A. Organización y creación de objetos

1. Proyecto, package e import

- Las clases están en dos paquetes declarados con package al inicio de cada archivo: modelo (los empleados) y aplicacion (Main).
- Main usa import para ArrayList, Scanner, Locale y para las clases del paquete modelo.
- **Por qué:** separar los datos del negocio (modelo) de la interacción con el usuario (aplicacion).

2. Clase, objeto, instancia e instanciación

Cada clase está en su propio archivo .java. En registrarEmpleadoFijo() se crea una instancia independiente por cada empleado:

```
EmpleadoFijo nuevo = new EmpleadoFijo(dpi, nombre, sueldo, bonificacion);
empleados.add(nuevo);
```

3. Constructores, parámetros, argumentos y this

```
public Empleado(String dpi, String nombre) {
    this.dpi = dpi;
    this.nombre = nombre;
}
```

- **Parámetros:** dpi y nombre. Son las variables que el constructor declara entre paréntesis para recibir datos.
- **Argumentos:** los valores concretos que se envían al llamarlo. Para Ana, los argumentos son "1234567890101" y "Ana López" (en Main, las variables dpi y nombre capturadas en consola).
- **this:** this.dpi es el atributo del objeto; dpi (sin this) es el parámetro. this los distingue.
- Los constructores no declaran tipo de retorno.

B. Encapsulación, abstracción y herencia

4. Atributos y modificadores de acceso

- Todos los atributos son private: dpi, nombre, sueldoMensual, bonificacion, pagoPorHora, horasTrabajadas y, en Main, meses, empleados, scanner y numeroMes.
- Se consultan con getters (getDpi(), getNombre(), etc.). No se usaron setters porque los datos no se modifican después de crear el empleado.

- Los parámetros y las variables locales no llevan private.

5. Clase y método abstracto

- Empleado es abstract y declara calcularPago() y obtenerTipo() sin cuerpo. No se puede hacer new Empleado(...).
- **Por qué:** la base guarda lo común (DPI y nombre) y obliga a cada clase hija a definir su propio cálculo.

6. Herencia y sobrescritura

- EmpleadoFijo y EmpleadoPorHora usan extends Empleado e inicializan DPI y nombre con super(dpi, nombre); no repiten esos atributos.
- Sobrescriben calcularPago() con @Override:

```
EmpleadoFijo:     return sueldoMensual + bonificacion;
EmpleadoPorHora:  return pagoPorHora * horasTrabajadas;
```

7. Polimorfismo

Ambos tipos se guardan en una sola colección, ArrayList<Empleado>. En imprimirNomina() se recorre con referencias de tipo Empleado:

```
for (Empleado e : empleados) {
    double pago = e.calcularPago();
    total += pago;
}
```

La clase real del objeto decide qué fórmula se ejecuta. El reporte no usa if, switch ni instanceof para elegirla.

C. Arreglo simple y ArrayList

8. String[] meses (arreglo simple)

Declarado en Main con los 12 nombres predefinidos. Su tamaño es fijo, por eso se usa un arreglo.

9. meses[numeroMes - 1]

Se usa en todos los títulos:

```
System.out.println("NÓMINA DEL MES: " + meses[numeroMes - 1].toUpperCase());
```

Se resta 1 porque los índices empiezan en 0: si se ingresa 1, meses[0] es "Enero"; si se ingresa 3, meses[2] es "Marzo"; si se ingresa 12, meses[11] es "Diciembre". numeroMes se valida entre 1 y 12 en leerOpcion() antes de usarse como índice, así nunca se accede fuera del arreglo.

10. ArrayList<Empleado>

Se usa porque la cantidad de empleados crece mientras se ingresan.

- **add():** empleados.add(nuevo) guarda al empleado; el contador sube solo después de guardarlo.
- **size():** empleados.size() da el valor de "Trabajadores actuales" y "Total de trabajadores".
- **Recorrido:** for (Empleado e : empleados) en imprimirNomina().

D. Métodos, ámbito y alcance

11. Métodos y main

- main es el punto de entrada y delega en seleccionarMes(), mostrarMenuPrincipal(), ingresarNomina(), registrarEmpleadoFijo(), registrarEmpleadoPorHora(), imprimirNomina(), limpiarPantalla() y pausar().
- Métodos con parámetros: leerOpcion(String mensaje, int min, int max), leerMonto(String mensaje) y formato(double valor).
- Métodos que devuelven resultado: leerOpcion() devuelve int; leerDpi() y leerNombre() devuelven String; leerMonto() y calcularPago() devuelven double.

12. Ámbito y alcance

- **Atributo:** nombre en Empleado. Es private; se usa en toda la clase y, desde otras clases, solo mediante getNombre().
- **Parámetro:** mensaje en leerMonto(String mensaje). Solo existe dentro de ese método.
- **Variable local:** pago dentro del for de imprimirNomina(). Solo existe dentro de ese bloque.

E. Validaciones

- Mes y menús: entero dentro del rango permitido (leerOpcion).
- DPI: String con exactamente 13 dígitos (leerDpi, expresión \d{13}).
- Nombre: no vacío ni solo espacios (leerNombre, trim().isEmpty()).
- Sueldo, bonificación, pago por hora y horas: número finito mayor o igual a 0 (leerMonto, con Double.parseDouble en try/catch y Double.isFinite).
- Si el dato es inválido se explica el error y se pide de nuevo ese dato. El empleado se crea solo cuando todos sus datos son válidos.

F. Demostración del caso de la sección 11

Entradas en orden: 3, 1, 1, 1234567890101, Ana López, 4000, 300, 1, 2, 9876543210101, Luis Pérez, 25, 120, 2, 2.

Resultado al registrar:

```
Ingrese el mes a trabajar (1-12): 3
NÓMINA DEL MES: MARZO                    (meses[2])
Pago del mes = sueldo mensual + bonificación
Pago de Ana López = Q 4300.00
Empleado agregado correctamente.
Trabajadores actuales = 1

Pago del mes = pago por hora x horas trabajadas
Pago de Luis Pérez = Q 3000.00
Empleado agregado correctamente.
Trabajadores actuales = 2
```

Reporte (opción 2 del menú):

```
NÓMINA DEL MES: MARZO
DPI             Nombre                  Tipo           Pago (Q)
```

```
1234567890101   Ana López             Fijo            4300.00
9876543210101   Luis Pérez            Por hora        3000.00

Total de trabajadores: 2
Total de nómina: Q 7300.00
```

Otras pruebas realizadas:

- Mes 1 muestra Enero y mes 12 muestra Diciembre.
- Mes 0, 13 o texto: se rechaza y se vuelve a pedir; nunca se usa como índice.
- Reporte sin empleados: muestra "No hay empleados registrados en la nómina de este mes", cantidad 0 y total Q 0.00.
- Opción de menú inválida (por ejemplo 7): muestra error y vuelve a pedir.
- DPI de 3 dígitos, nombre vacío, texto ("abc") y valor negativo (-5) en campos numéricos: muestran error y piden de nuevo el dato sin cerrar el programa.
- Consultar el reporte varias veces no borra ni duplica empleados.